

Implementação de CRUD de Clientes com Flask

Requisitos

Ferramentas:

- VS Code
 - **Extensões:**
 - SQLite
 - SQLite Viewer
 - SQLite3 Editor
 - Live Server
 - Python
- Python
- DB Browser

Bibliotecas:

- flask
 - flask-sqlalchemy
-

Etapas

1. Crie o arquivo `app.py`

2. Instale o Flask:

```
pip install flask
```

3. Instale o Flask-SQLAlchemy:

```
pip install flask-sqlalchemy
```

4. Importe o Flask:

```
from flask import Flask
```

5. Crie a aplicação Flask:

```
app = Flask(__name__)
```

6. Crie a rota padrão:

```
@app.route('/')
def index():
    return 'Olá Mundo!'
```

7. Crie o inicializador do servidor:

```
if __name__ == '__main__':
    app.run(debug=True, port=5153)
```

8. Importe o `render_template` do Flask:

```
from flask import Flask, render_template
```

9. Crie a pasta `templates`

10. Crie o arquivo `index.html` na pasta `templates`

11. Altere a rota padrão para incluir o `render_template`:

```
@app.route('/')
def index():
    return render_template('index.html')
```

12. No arquivo `index.html`, crie o HTML padrão:

- `html lang="pt-BR"`
- `title: PetShop`
- Crie um `<h1>PetShop Au Au</h1>`

13. Aprendendo a interação do Python com HTML:

- a. Crie uma lista simples:

```
clientes = ['cliente 1', 'cliente 2', 'cliente 3', 'cliente 4']
```

- b. Inclua os clientes como parâmetro no `render_template` da rota padrão.

```
return render_template('index.html', clientes=clientes)
```

14. No arquivo `index.html`, renderize os clientes criando um laço de repetição Jinja:

```
<h2>Clientes Cadastrados</h2>
<ul>
    {% for cliente in clientes %}
        <li>{{ cliente }}</li>
    {% endfor %}
</ul>
```

15. Compreendida a interação do Python com HTML, passaremos a verificar a interação do Python com o banco de dados.

16. Importe a biblioteca `flask_sqlalchemy`:

```
from flask_sqlalchemy import SQLAlchemy
```

17. Instancie o Banco de Dados:

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///petshop_db.db'
db = SQLAlchemy(app)
```

O sistema cria automaticamente a pasta `instances` por padrão.

18. Crie a classe para gerenciar a tabela `CLIENTE`:

```
class CLIENTE(db.Model):
    id_cliente = db.Column(db.Integer, primary_key=True, autoincrement=True)
    nome = db.Column(db.String(100), nullable=False)
    telefone = db.Column(db.String(15), unique=True, nullable=False)
    email = db.Column(db.String(100), unique=True)
    cpf = db.Column(db.String(14), unique=True)
    data_cadastro = db.Column(db.DateTime(), default=datetime.datetime.now)
```

Lembre-se de importar a biblioteca `datetime`.

19. Atualize o inicializador do servidor para criar o banco de dados:

```
with app.app_context():
    db.create_all()
```

Implementação do CRUD

1. Read (Ler)

a. Na rota padrão, altere a variável e insira a consulta:

```
clientes = CLIENTE.query.all()
```

b. Verifique o resultado na página.

c. Para visualizar os dados, modifique o objeto no `index.html`:

```
{{ cliente.nome }}
```

2. Create (Criar)

a. Importe os métodos `request` e `redirect` do Flask.

```
from flask import Flask, render_template, request, redirect
```

b. Implemente a rota para criar um novo cliente:

```
@app.route('/create', methods=['POST'])
```

c. Crie a função `create_cliente`:

```
def create_cliente():
```

d. Faça a ligação dos dados com o formulário:

```
nome = request.form['nome']
telefone = request.form['telefone']
email = request.form['email']
cpf = request.form['cpf']
```

e. Crie a variável `novo_cliente`:

```
novo_cliente = CLIENTE(nome=nome, telefone=telefone, email=email, cpf=cpf)
```

f. Abra uma sessão para adicionar o cliente no banco de dados:

```
db.session.add(novo_cliente)
```

g. Confirme a alteração no banco de dados com um commit:

```
db.session.commit()
```

h. Finalize redirecionando para a página inicial:

```
return redirect('/')
```

2.1 Atualizando o `index.html` com o formulário

Adicione o formulário no arquivo `index.html`:

```
<h2>Adicionar Novo Cliente</h2>
<form action="/create" method="POST">
  <input type="text" name="nome" placeholder="Nome" required>
  <input type="text" name="telefone" placeholder="Telefone" required>
  <input type="text" name="email" placeholder="E-mail">
  <input type="text" name="cpf" placeholder="CPF">
  <button type="submit">Cadastrar Cliente</button>
</form>
```

2.2 Tratamento de erro de integridade

Para evitar erros ao cadastrar um cliente com dados duplicados, adicione uma validação:

```
# Valida se o cliente já está cadastrado
existe_cliente_telefone = CLIENTE.query.filter_by(telefone=telefone).first()
existe_cliente_email = CLIENTE.query.filter_by(email=email).first()
existe_cliente_cpf = CLIENTE.query.filter_by(cpf=cpf).first()

if (existe_cliente_telefone or existe_cliente_email or existe_cliente_cpf):
    return 'ERRO: Cliente já Cadastrado!', 400
```

3. Delete (Apagar)

a. Implemente a rota para deletar um cliente:

```
@app.route('/delete/<int:id_cliente>', methods=['POST'])
```

b. Crie a função para deletar o cliente:

```
def delete_cliente(id_cliente):
```

c. Procure o cliente no banco de dados pelo ID:

```
cliente = CLIENTE.query.get(id_cliente)
```

d. Verifique se o cliente existe e, em caso positivo, faça a exclusão:

```
if cliente:
    db.session.delete(cliente)
    db.session.commit()
```

3.1 Atualizando o index.html para o Delete

Modifique a lista de clientes para incluir o botão de exclusão:

```
<ul>
    {% for cliente in clientes %}
        <li>
            {{ cliente.nome }}
            <form action="/delete/{{ cliente.id_cliente }}" method="POST" style="display: inline;">
                <button type="submit">Excluir</button>
            </form>
        </li>
    {% endfor %}
</ul>
```

4. Update (Atualizar)

a. Implemente a rota para atualizar um cliente:

```
@app.route('/update/<int:id_cliente>', methods=['POST'])
```

b. Crie a função para atualizar o cliente:

```
def update_cliente(id_cliente):
```

c. Procure o cliente no banco de dados pelo ID:

```
cliente = CLIENTE.query.get(id_cliente)
```

d. Verifique se o cliente existe e, em caso positivo, faça a atualização:

```
if cliente:
    cliente.nome = request.form['nome']
    cliente.telefone = request.form['telefone']
    cliente.cpf = request.form['cpf']
    db.session.commit()
```

4.1 Atualizando o index.html para o Update

Atualize a lista para incluir os campos de atualização e o botão de exclusão:

```
<ul style="list-style: none;">
  {% for cliente in clientes %}
    <li>
      <form action="/update/{{ cliente.id_cliente }}" method="POST" style="display: inline;">
        <input type="text" name="nome" value="{{ cliente.nome }}">
        <input type="text" name="telefone" value="{{ cliente.telefone }}">
        <input type="text" name="email" value="{{ cliente.email }}">
        <input type="text" name="cpf" value="{{ cliente.cpf }}">
        <button type="submit">Atualizar</button>
      </form>

      <form action="/delete/{{ cliente.id_cliente }}" method="POST" style="display: inline;">
        <button type="submit">Excluir</button>
      </form>
    </li>
  {% endfor %}
</ul>
```